



Courses : Advanced Web Programming (PWL)
Courses : D4 – Informatics Engineering / D4 – Business Information Systems
Semester : 4 (four) / 5 (five)
The meeting : 7 (seven)

JOBSHEET 07

Authentication and Authorization in Laravel

Laravel Authentication is used to protect pages or features of the web that are only accessed by certain authorized persons. Features like this are usually found in systems that have administrator features or systems that have users who can add data.

Laravel makes implementing authentication very simple and has provided a variety of features that can be leveraged without the need to install specific modules. The authentication configuration file is located in `config/auth.php`, which contains several well-documented options for changing the configuration of the authentication service.

At its core, Laravel's authentication facility consists of "*guards*" and "*providers*". *Guards* specify how the user is authenticated for each request. For example, Laravel sends with *guards* to a session by using session storage and cookies.

Middleware

The middleware is the intermediary layer between *the incoming* HTTP route request and the *action* of the Controller to be executed. **Middleware** allows us to nail various tasks both before and after the action is performed. We can also use the *CLI tool* to create a **middleware** in **Laravel**. Some examples of using **middleware** include authentication, validation, request manipulation, and more. Below are the benefits of **Middleware**:

- **Security** : in **Middleware** it allows us to verify if the user is authenticated before accessing a certain page. Thus, we can protect sensitive data and control user access rights.
- **Data Filtering**: **Middleware** can be used to manipulate request data before an *action* in *the controller* is performed. For example, we can first check the data sent by the user before the data is further processed or we want to modify the data to be sent and then we can double-check the data that will be sent by the user before the data is processed.



- **Logging and Auditing : Middleware** can also be used to log user activity or audit incoming requests. This can help in the monitoring and analysis of the application.

INFO

We'll use Laravel Auth manually like

<https://laravel.com/docs/10.x/authentication#authenticating-users>

In accordance with **Case Study PWL.pdf**. So our Laravel 10 project is still the same using **the PWL_POS repository**.

We will use **Project PWL_POS** until the 12th meeting, as the project we will learn

A. Manual Authentication Implementation in Laravel

Authentication is the process of verifying the identity of a user trying to access a system. In the context of a web application, authentication ensures that users who try to log in have the appropriate access rights based on credentials such as email and password. The authentication process is different from **authorization**, which is an advanced step to determine what access rights a user has after they have been successfully authenticated.

Authentication Concepts in Laravel

Laravel offers a very flexible authentication system. Laravel provides a built-in authentication mechanism through authentication scaffolding services such as Laravel *Jetstream* and *Breeze*, which can automatically generate authentication pages and logic. However, sometimes developers need a more manual authentication implementation to give them full control over every aspect of the process.

Some of the essential components in the Laravel authentication system include:

- **Guard**: A component that governs how users are authenticated for each request. *The default guard* uses session and cookies.
- **Provider**: Governs how users are retrieved from databases or other data sources. *The default provider* retrieves user data from the database by using the Eloquent ORM.
- **Session**: Laravel uses a session to store the user's authentication state. Sessions allow the system to remember which users are already logged in between different HTTP requests.



Common flows of authentication include:

1. *Login*: The user submits credentials (usually an email and password).
2. *Credential Verification*: The system checks if the credentials provided match the data in the database.
3. *Session Creation*: If the credentials are correct, the system will create a session for the user that will be stored on the server.
4. *Access to Protected Pages*: Authenticated users can access pages protected by *auth middleware*.
5. *Logout*: Users can log out of the system and their session will be deleted.

Middleware Authentication

Middleware auth in Laravel is used to protect routes or pages so that they can only be accessed by authenticated users. If a user tries to access a route that requires authentication without signing in, they will be redirected to the login page.

- **The guard** is responsible for handling the user authentication process. Laravel uses session-based guards by default for web authentication, but it also supports token-based guards (such as APIs).
- **The provider** is in charge of retrieving the user from the database. Laravel provides a *default provider* that uses Eloquent, but also supports other *providers* such as Query Builder.

Implementation in Laravel 10

We'll implement the use of authentication in Laravel. In this application, we will try to create authentication in Laravel, so that we understand the steps in creating Authentication

Practicum 1 – Authentication Implementation:

1. We open our PWL_POS laravel project , and we modify our app's configuration in `config/auth.php`

```
62     'providers' => [  
63         'users' => [  
64             'driver' => 'eloquent',  
65             'model' => App\Models\User::class,  
66         ],
```



In this section we adjust the Model for the m_user table that we have created

```
62         'providers' => [  
63             'users' => [  
64                 'driver' => 'eloquent',  
65                 'model' => App\Models\UserModel::class,  
66             ],  
67         ],
```

2. Next, we modify the UserModel.php a little to be able to carry out the authentication process

```
<?php  
namespace App\Models;  
  
use Illuminate\Database\Eloquent\Model;  
use Illuminate\Database\Eloquent\Factories\HasFactory;  
use Illuminate\Database\Eloquent\Relations\BelongsTo;  
use Illuminate\Foundation\Auth\User as Authenticatable; // implementasi class Authenticatable  
  
class UserModel extends Authenticatable  
{  
    use HasFactory;  
  
    protected $table = 'm_user';  
    protected $primaryKey = 'user_id';  
    protected $fillable = ['username', 'password', 'nama', 'level_id', 'created_at', 'updated_at'];  
  
    protected $hidden = ['password']; // jangan di tampilkan saat select  
  
    protected $casts = ['password' => 'hashed']; // casting password agar otomatis di hash  
  
    /**  
     * Relasi ke tabel level  
     */  
    public function level(): BelongsTo  
    {  
        return $this->belongsTo(LevelModel::class, 'level_id', 'level_id');  
    }  
}
```

3. Next, we create a [AuthController.php](#) to process the login that we will do

```
<?php  
namespace App\Http\Controllers;  
  
use Illuminate\Http\Request;  
use Illuminate\Support\Facades\Auth;  
  
class AuthController extends Controller  
{  
    public function login()  
    {  
        if(Auth::check()){ if you are logged in, then redirect to the home page  
            return redirect('/');  
        }  
    }  
}
```



```
}  
    return view('auth.login');  
}  
  
public function postlogin(Request $request)  
{  
    if($request->ajax() || $request->wantsJson()){  
        $credentials = $request->only('username', 'password');  
  
        if (Auth::attempt($credentials)) {  
            return response()->json([  
                'status' => true,  
                'message' => 'Login Successful', 'redirect' => url('/')  
            ]);  
        }  
  
        return response()->json([  
            'status' => false,  
            'message' => 'Login Failed'  
        ]);  
    }  
  
    return redirect('login');  
}  
  
public function logout(Request $request)  
{  
    Auth::logout();  
  
    $request->session()->invalidate();  
    $request->session()->regenerateToken();  
    return redirect('login');  
}  
}
```

4. Once we have created the `AuthController.php`, we create a view to display the login page. The view we create in `auth/login.blade.php`, we can take the login view from the login example in the **AdminLTE** template as follows (in this login example, we use the **V2 login page** in **AdminLTE**)

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="utf-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1">  
    <title>User Login</title>  
  
    <!-- Google Font: Source Sans Pro -->  
    <link rel="stylesheet"  
href="https://fonts.googleapis.com/css?family=Source+Sans+Pro:300,400,400i,700&display=fallb  
ack">  
    <!-- Font Awesome -->  
    <link rel="stylesheet" href="{{ asset('plugins/fontawesome-free/css/all.min.css') }}">  
    <!-- icheck bootstrap -->  
    <link rel="stylesheet" href="{{ asset('plugins/icheck-bootstrap/icheck-bootstrap.min.css')  
}}">  
    <!-- SweetAlert2 -->  
    <link rel="stylesheet" href="{{ asset('plugins/sweetalert2-theme-bootstrap-4/bootstrap-  
4.min.css') }}">  
    <!-- Theme style -->  
    <link rel="stylesheet" href="{{ asset('dist/css/adminlte.min.css') }}">  
</head>  
<body class="hold-transition login-page">  
<div class="login-box">
```



```
<!-- /.login-logo -->
<div class="card card-outline card-primary">
  <div class="card-header text-center"><a href="{{ url('/') }}"
class="h1"><b>Admin</b>LTE</a></div>
  <div class="card-body">
    <p class="login-box-msg">Sign in to start your session</p>
    <form action="{{ url('login') }}" method="POST" id="form-login">
@csrf
      <div class="input-group mb-3">
        <input type="text" id="username" name="username" class="form-control"
placeholder="Username">
        <div class="input-group-append">
          <div class="input-group-text">
            <span class="fas fa-envelope"></span>
          </div>
        </div>
        <small id="error-username" class="error-text text-danger"></small>
      </div>
      <div class="input-group mb-3">
        <input type="password" id="password" name="password" class="form-control"
placeholder="Password">
        <div class="input-group-append">
          <div class="input-group-text">
            <span class="fas fa-lock"></span>
          </div>
        </div>
        <small id="error-password" class="error-text text-danger"></small>
      </div>
      <div class="row">
        <div class="col-8">
          <div class="icheck-primary">
            <input type="checkbox" id="remember"><label for="remember">Remember Me</label>
          </div>
        </div>
        <!-- /.col -->
        <div class="col-4">
          <button type="submit" class="btn btn-primary btn-block">Sign In</button>
        </div>
        <!-- /.col -->
      </div>
    </form>
  </div>
<!-- /.card-body -->
</div>
<!-- /.card -->
</div>
<!-- /.login-box -->

<!-- jQuery -->
<script src="{{ asset('plugins/jquery/jquery.min.js') }}"></script>
<!-- Bootstrap 4 -->
<script src="{{ asset('plugins/bootstrap/js/bootstrap.bundle.min.js') }}"></script>
<!-- jquery-validation -->
<script src="{{ asset('plugins/jquery-validation/jquery.validate.min.js') }}"></script>
<script src="{{ asset('plugins/jquery-validation/additional-methods.min.js') }}"></script>
<!-- SweetAlert2 -->
<script src="{{ asset('plugins/sweetalert2/sweetalert2.min.js') }}"></script>
<!-- AdminLTE App -->
<script src="{{ asset('dist/js/adminlte.min.js') }}"></script>

<script>
$.ajaxSetup({
  headers: {
    'X-CSRF-TOKEN': $('meta[name="csrf-token"]').attr('content')
  }
});
```



```
$(document).ready(function() {
    $("#form-login").validate({
    rules: {
        username: {required: true, minlength: 4, maxlength: 20},
        password: {required: true, minlength: 6, maxlength: 20}
    },
    submitHandler: function(form) { when valid, then the part that will be executed
    $.ajax({
        url: form.action,
        type: form.method,
        data: $(form).serialize(),
        success: function(response) {
            if(response.status){ if successful
        Swal.fire({
            icon: 'success',
            title: 'Succeed',
            text: response.message,
        }).then(function() {
        window.location = response.redirect;
        });
        }else{ jika error
        $('.error-text').text('');
        $.each(response.msgField, function(prefix, val) {
        $('#error-'+prefix).text(val[0]);
        });
        Swal.fire({
            icon: 'error',
            title: 'Something Went Wrong',
            text: response.message
        });
        }
        }
        });
        return false;
    },
    errorElement: 'span',
    errorPlacement: function (error, element) {
        error.addClass('invalid-feedback');
        element.closest('.input-group').append(error);
    },
    highlight: function (element, errorClass, validClass) {
        $(element).addClass('is-invalid');
    },
    unhighlight: function (element, errorClass, validClass) {
        $(element).removeClass('is-invalid');
    }
    });
});
</script>
</body>
</html>
```

5. Then we modify the `route/web.php` so that all routes are included in the auth



```
<?php

use App\Http\Controllers\UserController;
use App\Http\Controllers\SupplierController;
use App\Http\Controllers\BarangController;
use App\Http\Controllers\AuthController;
use App\Http\Controllers\KategoriController;
use App\Http\Controllers\LevelController;
use App\Http\Controllers>WelcomeController;
use Illuminate\Support\Facades\Route;

Route::pattern('id','[0-9]+'); // artinya ketika ada parameter {id}, maka harus berupa angka

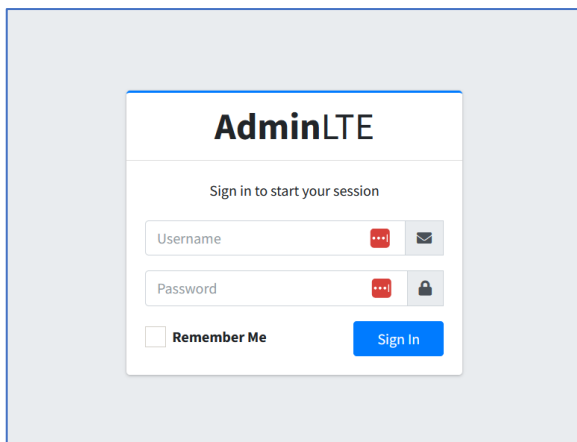
Route::get('login', [AuthController::class,'login'])->name('login');
Route::post('login', [AuthController::class,'postlogin']);
Route::get('logout', [AuthController::class,'logout'])->middleware('auth');

Route::middleware(['auth'])->group(function(){ // artinya semua route di dalam group ini harus login dulu

    // masukkan semua route yang perlu autentikasi di sini

});
```

6. When we try to access the localhost/PWL_POS/public page, a start page will appear to log in to the application



Task 1 – Authentication Implementation:

1. Please implement the login process on your respective projects
2. Please implement the logout process on the web page you created
3. Observe and describe each stage you worked on, and describe it in the report
4. Submit the code for the Authentication implementation on your github repository.



B. Authorization Implementation in Laravel

Authorization is the process after authentication is successfully carried out (in other words, we successfully log in to the system). *Authorization* relates to the user's access rights in using the system. *Authorization* grants/ensures our access rights (access permissions), in accordance with the rules (roles) that exist in the system. *Authorization* is very important to limit user access according to its designation.

For example, when we access the LMS with an account (*username* and *password*) that is a student. When we succeed in authentication, our access rights will also be given like students. Such as viewing courses, viewing materials, downloading material files, doing/uploading assignments, taking exams, etc. We will not be given access rights by the system to create materials, create exam questions, create assignments, give assignment grades because these access rights enter the realm of **Lecturer/Lecturer** type accounts.

In addition to providing built-in authentication services, Laravel also provides a simple way to authorize user actions against specific resources. For example, even if a user is authenticated, they may not be authorized to update or delete certain Eloquent models or database records managed by your app. Laravel's authorization feature provides an easy and organized way to manage these types of authorization checks.

Practicum 2 – Implementing *Authorizations* in Laravel with Middleware

We'll apply authorization to the Laravel project using Middleware as an access check. The steps we do are as follows:

1. We modify **UserModel.php** by adding the following code

```
/**
 * Relasi ke tabel level
 */
public function level(): BelongsTo
{
    return $this->belongsTo(LevelModel::class, 'level_id', 'level_id');
}

/**
 * Mendapatkan nama role
 */
public function getRoleName(): string
{
    return $this->level->level_nama;
}

/**
 * Cek apakah user memiliki role tertentu
 */
public function hasRole($role): bool
{
    return $this->level->level_kode == $role;
}
```



2. Then we create *middleware* with the name `AuthorizeUser.php`. We can create *middleware* by typing commands in terminal/CMD

```
php artisan make:middleware AuthorizeUser
```

The *middleware* file will be created in `app/Http/Middleware/AuthorizeUser.php`

3. Then we edit the *middleware* `AuthorizeUser.php` to be able to check whether the user accessing has the appropriate Level/Role/Group

```
1  <?php
2  namespace App\Http\Middleware;
3
4  use Closure;
5  use Illuminate\Http\Request;
6  use Symfony\Component\HttpFoundation\Response;
7
8  class AuthorizeUser
9  {
10     /**
11      * Handle an incoming request.
12      *
13      * @param  \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response)
14      */
15     public function handle(Request $request, Closure $next, $role = ''): Response
16     {
17         $user = $request->user(); // ambil data user yg login
18         // fungsi user() diambil dari UserModel.php
19         if($user->hasRole($role)){ // cek apakah user punya role yg diinginkan
20             return $next($request);
21         }
22         // jika tidak punya role, maka tampilkan error 403
23         abort(403, 'Forbidden. Kamu tidak punya akses ke halaman ini');
24     }
25 }
```

4. Let's register it to `app/Http/Kernel.php` for the *middleware* we created just now

```
protected $middlewareAliases = [
    'auth' => \App\Http\Middleware\Authenticate::class,
    'authorize' => \App\Http\Middleware\AuthorizeUser::class, // middleware yg kita buat
    'auth.basic' => \Illuminate\Auth\Middleware\AuthenticateWithBasicAuth::class,
    'auth.session' => \Illuminate\Session\Middleware\AuthenticateSession::class,
```

5. Now we look at the `m_level` table which is the table to store the level/group/role of the user exists

level_id	level_code	level_nama	created_at	updated_at	deleted_at
1	ADM	Administrator	NULL	NULL	NULL
2	MNG	Manager	NULL	NULL	NULL
3	STF	Staf	NULL	2024-08-16 01:49:20	NULL
4	KSR	Kasir	NULL	NULL	NULL



6. To try the *authorization* that we have created, we need to modify the `route/web.php` to determine which routes will be given access rights according to the user level

```
Route::middleware(['auth'])->group(function(){ // artinya semua route di dalam group ini harus login dulu
    Route::get('/', [WelcomeController::class, 'index']);
    // route Level

    // artinya semua route di dalam group ini harus punya role ADM (Administrator)
    Route::middleware(['authorize:ADM'])->group(function(){
        Route::get('/level', [LevelController::class, 'index']);
        Route::post('/level/list', [LevelController::class, 'list']); // untuk list json datatables
        Route::get('/level/create', [LevelController::class, 'create']);
        Route::post('/level', [LevelController::class, 'store']);
        Route::get('/level/{id}/edit', [LevelController::class, 'edit']); // untuk tampilkan form edit
        Route::put('/level/{id}', [LevelController::class, 'update']); // untuk proses update data
        Route::delete('/level/{id}', [LevelController::class, 'destroy']); // untuk proses hapus data
    });

    // route Kategori
```

In the code marked in red, there is `authorize:ADM`. The `ADM` code is the value of the `level_code` on the `m_level` table. This means that users who can access the route to manage data levels, are users who have a level as an Administrator.

7. To prove it, now we try to log in using an account other than the administrator level, and we access the route menu of that level

Task 2 – Implementation of Authoriization:

1. What do you understand in this practicum 2?
2. Observe and describe each stage you worked on, and describe it in the report
3. Submit the code for the Authorization implementation on your github repository.



C. Multi-Level Authorization in Laravel

What if there are levels/groups/roles with each other with the same access rights. For a simple example, Admin and Manager level users can both access the Item menu in the application we create. So it is impossible for us to create a route for each user level. This will take a lot of time, and a long process.

```
// artinya semua route di dalam group ini harus punya role ADM (Administrator)
Route::middleware(['authorize:ADM'])->group(function(){
    Route::get('/barang', [BarangController::class, 'index']);
    Route::post('/barang/list', [BarangController::class, 'list']);
    Route::get('/barang/create_ajax', [BarangController::class, 'create_ajax']); // ajax form create
    Route::post('/barang_ajax', [BarangController::class, 'store_ajax']); // ajax store
    Route::get('/barang/{id}/edit_ajax', [BarangController::class, 'edit_ajax']); // ajax form edit
    Route::put('/barang/{id}/update_ajax', [BarangController::class, 'update_ajax']); // ajax update
    Route::get('/barang/{id}/delete_ajax', [BarangController::class, 'confirm_ajax']); // ajax form confirm delete
    Route::delete('/barang/{id}/delete_ajax', [BarangController::class, 'delete_ajax']); // ajax delete
});

// artinya semua route di dalam group ini harus punya role MNG (Manager)
Route::middleware(['authorize:MNG'])->group(function(){
    Route::get('/barang', [BarangController::class, 'index']);
    Route::post('/barang/list', [BarangController::class, 'list']);
    Route::get('/barang/create_ajax', [BarangController::class, 'create_ajax']); // ajax form create
    Route::post('/barang_ajax', [BarangController::class, 'store_ajax']); // ajax store
    Route::get('/barang/{id}/edit_ajax', [BarangController::class, 'edit_ajax']); // ajax form edit
    Route::put('/barang/{id}/update_ajax', [BarangController::class, 'update_ajax']); // ajax update
    Route::get('/barang/{id}/delete_ajax', [BarangController::class, 'confirm_ajax']); // ajax form confirm delete
    Route::delete('/barang/{id}/delete_ajax', [BarangController::class, 'delete_ajax']); // ajax delete
});
```

This is an obstacle when we want to change access rights, then we will replace most of the routes that we have written. For this reason, we need to manage middleware so that we can support the dynamic addition of access rights.

Practicum 3 – Multi-Level Authorizaton Implementation in Laravel with Middleware

We'll implement multi-level authorization in the Laravel project using Middleware as an access check. The steps we do are as follows:

1. We modify `UserModel.php` to get `level_kode` from users who have logged in. So we create a function with the name `getRole()`



```
/**
 * Mendapatkan nama role
 */
public function getRoleName(): string
{
    return $this->level->level_nama;
}

/**
 * Cek apakah user memiliki role tertentu
 */
public function hasRole($role): bool
{
    return $this->level->level_kode == $role;
}

/**
 * Mendapatkan kode role
 */
public function getRole()
{
    return $this->level->level_kode;
}
```

2. Next, we modify the AuthorizeUser.php middleware with the following code

```
1  <?php
2  namespace App\Http\Middleware;
3
4  use Closure;
5  use Illuminate\Http\Request;
6  use Symfony\Component\HttpFoundation\Response;
7
8  class AuthorizeUser
9  {
10     /**
11      * Handle an incoming request.
12      *
13      * @param  \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFoundation\Response)
14      */
15     public function handle(Request $request, Closure $next, ... $roles): Response
16     {
17         $user_role = $request->user()->getRole(); // ambil data level_kode dari user yg login
18         if(in_array($user_role, $roles)){ // cek apakah level_kode user ada di dalam array roles
19             return $next($request); // jika ada, maka lanjutkan request
20         }
21         // jika tidak punya role, maka tampilkan error 403
22         abort(403, 'Forbidden. Kamu tidak punya akses ke halaman ini');
23     }
24 }
```

3. After that, we just need to fix the `route/web.php` adjust it to the desired role/level.
Example



```
// artinya semua route di dalam group ini harus punya role ADM (Administrator) dan MNG (Manager)
Route::middleware(['authorize:ADM,MNG'])->group(function(){
    Route::get('/barang',[BarangController::class,'index']);
    Route::post('/barang/list',[BarangController::class,'list']);
    Route::get('/barang/create_ajax',[BarangController::class,'create_ajax']); // ajax form create
    Route::post('/barang_ajax',[BarangController::class,'store_ajax']); // ajax store
    Route::get('/barang/{id}/edit_ajax',[BarangController::class,'edit_ajax']); // ajax form edit
    Route::put('/barang/{id}/update_ajax',[BarangController::class,'update_ajax']); // ajax update
    Route::get('/barang/{id}/delete_ajax',[BarangController::class,'confirm_ajax']); // ajax form confirm
    Route::delete('/barang/{id}/delete_ajax',[BarangController::class,'delete_ajax']); // ajax delete
});
```

4. Now we can give menu/route access rights to several user levels

Task 3 – Implementation of Multi-Level Authorization:

1. Please implement multi-level authorization in your respective projects
2. Observe and describe each stage you worked on, and describe it in the report
3. Implement multi-level authorization for all User Levels/Types and Menus corresponding to User Levels/Types
4. Submit the code for the Authorization implementation on your github repository.

Task 4 – Implementation of Registration Form:

1. Please implement the form for user registration.
2. Screenshot of the results you worked on
3. Commit and push the results of your tasks to each of your github repos

*That's all, and happy learning ****